# 📓 Day 8 Notes ✏️

*Checkout and Orders (My Recap)*

★ Today = my store started saving REAL orders. My cart was merged into main, so I started a fresh branch. This was my first day changing both the client AND the server.

## 1 Starting Clean

```
cd Desktop\virtual_tryon
git checkout main
git pull
git checkout -b feature/checkout
cd client
npm install
cd ../server
npm install
```

New feature = new branch from the latest main. I install in BOTH folders, because both have their own packages.

## 2 What I Built

- Server: two new tables (orders and order_items), a route to place an order (POST /api/orders), and a route to read one (GET /api/orders/:id)
- Client: a Checkout page with a form (name, email, address) and an order summary
- A thank-you page at /order/:id showing the order number, items and total
- A Proceed to checkout button on the Cart page
- Clear error messages, and the Place order button is disabled while the request is sent

Payment is a DEMO. Nothing is charged, and the page says so.

## 3 Big Idea: The Server Calculates the Price

The browser can send ANYTHING, even a made-up price. If the server trusted it, someone could buy a jacket for 1 cent.

```
// simplified example
// WRONG: trust the price sent by the browser
// RIGHT: look it up in the database
const product = db.prepare(
  'SELECT price FROM products WHERE id = ?'
).get(productId)
```

The browser only sends WHICH product and HOW MANY. The server finds the real price and works out the total itself.

## 4 Big Idea: Database Transaction

One order needs several saves: the order itself, then every item. What if the order is saved but the items fail? We'd have a half-finished order.

```
// simplified idea
const saveOrder = db.transaction((order, items) => {
  // 1. save the order
  // 2. save every item
})
```

A transaction means ALL OR NOTHING. If any step fails, nothing is saved.

## 5 Validation and Status Codes

- The server checks the input: name, email and address not empty, items not empty, quantities are whole numbers of 1 or more
- 400 = the request was bad (for example an empty name). The server sends a clear message back.
- 404 = not found (for example an order number that doesn't exist)

The page shows the server's message instead of crashing.

## 6 Restart Both Servers

The server code changed, so the OLD server had to be stopped (Ctrl + C) and started again (node index.js). Otherwise I would test old code and think my feature was broken.

Orders are saved in MY laptop's database file (server/data/store.db). That file is not shared through git, so Fahad's laptop has its own orders.

## 7 Testing Checklist

- Place an order with the form EMPTY → an error, not a crash
- Fill the form and place the order → order page with an order number
- The cart is empty after ordering

- Open checkout with an empty cart → friendly message
- Refresh the order page → the order is still there (it was saved in the database)

## 8 Pushing

```
git status                    → changes in client AND server
git add .
git commit -m "Add checkout and orders"
git push -u origin feature/checkout
```

Fahad checked the branch on GitHub, pulled it, placed a test order on his own laptop, and merged it into main.

> 💡 **Big takeaway:**
> The server never trusts the browser, so it calculates prices itself. A transaction saves everything or nothing. Validation turns bad input into clear messages. After changing server code, always restart the server. Next: making the store look good.

*— made while building: AI Virtual Try-On E-commerce App —*